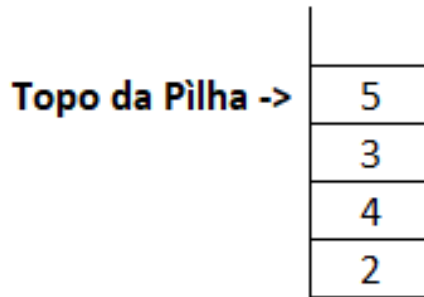


1. Cite dois exemplos práticos de **PILHA** ao qual utilizamos no dia a dia na área computacional.
2. Como funcionam as operações (inserção e remoção) em uma **PILHA** e de uma **FILA**? Explique.
3. Utilizando a linguagem c, apresente a estrutura mínima de um nó em uma **FILA**.
4. Em uma **PILHA** necessitamos definir uma variável de ponteiro independente dos demais dados contidos no nó. Qual deve ser o tipo de ponteiro para apontar para o próximo nó e quanto ele consumirá de memória? Justifique sua resposta.
5. Quais as possíveis operações que podemos efetuar em uma **PILHA**? Explique.
6. Em uma pilha, o que acontece se atribuirmos NULL para o ponteiro que aponta para o topo da **PILHA**?
7. Desenvolva um programa em C que simule uma **PILHA** com 20 valores aleatórios entre 10 e 125. Posteriormente, remova todos os elementos ímpares da pilha e apresente a pilha final ao usuário. Não esqueça de liberar a pilha toda ao encerrar o programa.
8. Com base no exercício anterior, faça o mesmo para uma **FILA**.
9. Utilizando a linguagem c, apresente a estrutura mínima de um nó em uma **PILHA**.
10. Construa um programa em linguagem C que implemente uma **FILA** contendo o nome e idade de 10 pessoas fornecidas pelo usuário. Posteriormente, monte mais duas filas (FILA2 e FILA3) onde a primeira deverá conter as pessoas até 30 anos e a segunda acima de 30. Apresente as duas novas filas ao usuário e posteriormente encerre o programa, liberando a memória das três filas. Obs. A fila inicial deverá ser esvaziada ao final do processo.
11. Desenvolva um programa denominado **PILHA1** em linguagem C que implemente as operações de uma pilha (pop, push e imprimir) cujo nó deverá conter nome[30] e idade. O usuário deverá ter a opção de empilhar, desempilhar, mostrar pilha e sair.
12. Construa um programa em linguagem c que monte uma pilha de 15 elementos com valores aleatórios (não repetidos) entre 10 e 100.

Posteriormente, monte duas pilhas (pares e ímpares), distribuindo e esvaziando a pilha original.

13. Considere a seguinte estrutura de dados do tipo Pilha, na qual existem quatro valores armazenados e cujo topo é indicado pelo ponteiro Topo da pilha.



A sequência de instruções expressas a seguir na forma de um pseudocódigo deve ser executada com base no estado atual da PILHA.

As instruções PUSH e POP são instruções típicas de estruturas de dados do tipo Pilha, e representam, respectivamente, inserção e exclusão.

1. Soma = 0;
2. POP(x);
3. Soma = Soma + x;
4. x = 10;
5. PUSH(x);
6. x = 12;
7. PUSH(x);
8. POP(x);
9. POP(x);
10. Soma = Soma + x;

Com base nessa sequência de instruções, apresente o valor final da variável soma.

14. Com base no exercício 12 (**PILHA ORIGINAL**), construa uma função que retorne a quantidade de elementos maiores que 50 na pilha.

15. Utilizando linguagem C, construa um programa que possua um menu (inserir, remover, imprimir e sair) e contemple uma **FILA** contendo nome e idade. O nome não poderá ter tamanho fixo, ou seja, deverá ser alocado dinamicamente.
16. Construa um programa em linguagem C que implemente uma **PILHA** através de 10 elementos (entre 10 e 100) gerados randomicamente (não repetidos). Posteriormente, apresente o conteúdo da pilha.
17. Dada a sequência de operações abaixo, indique o conteúdo final da **PILHA** (do topo para a base), supondo que ela se inicie vazia: push('A'), push('B'), pop(), push('C'), push('D'), pop(), pop(), push('E'), pop(), push('A'), push('Z'), push('K'), pop(), pop().
18. Construa uma **FILA** contendo 20 valores aleatórios entre 1 e 100. Posteriormente:
- remova os valores múltiplos de 5;
 - apresente a fila final.
19. Utilizando **PILHA**, desenvolva um programa que verifique se uma palavra é palíndromo.
20. Desenvolva um programa que leia uma expressão matemática e utilize **PILHA** para verificar se:
- parênteses;
 - colchetes;
 - chaves;
- estão balanceados.

Exemplo:

(A + B) * { C - D }